

AiSIO FIEMAP 논쟁 기술 분석: Christoph Hellwig vs. Samsung

출처: SNIA SDC StorageAI 2026 — AiSIO: Orchestrating Storage I/O Across CPUs and Accelerators

분석 대상: 강연 중 14:34 및 Q&A 26:35에 기록된 Christoph Hellwig의 이의 제기와 Samsung 측 답변

1. 논쟁의 경위

Samsung 연구원이 AiSIO의 오픈소스 컴포넌트를 소개하는 중, 청중석에서 Christoph Hellwig가 발언을 끊으며 이의를 제기했다. 이후 Q&A에서 더 상세한 비판을 이어갔다.

Christoph Hellwig의 발언 (14:34, 강연 중 인터럽트)

“is a proper way to do it and I told for two years to multiple members in your team how to do it and it’s really upsetting that you can keep publishing kind of dangerous”

자막 일부 누락이 있으나 문맥상 다음과 같이 읽힌다: “그렇게 하는 것이 올바른 방법이고, 저는 2년 동안 당신 팀의 여러 구성원에게 어떻게 해야 하는지 말했습니다. 위험한 것을 계속 발표하는 것은 정말 실망스럽습니다.”

Christoph Hellwig의 발언 (26:35, Q&A)

“debugging tool. It has different content for different file systems. It’s a massive risk for uh cost and corruption due to concurrent activity. That’s why I told everyone including your boss two years ago, you need to expand the PFS block layout layouts that have all the mechanisms to deal with that including relocation mechanism mechanism for defending clients”

번역: “그것은 디버깅 도구입니다. 파일 시스템마다 내용이 다릅니다. 동시 활동으로 인한 손상 위험이 매우 큼니다. 그래서 제가 2년 전에 당신 상사를 포함한 모든 사람에게 말했습니다. PFS 블록 레이아웃을 확장해야 한다고, 재배치 메커니즘을 포함해 클라이언트를 보호하는 모든 메커니즘을 갖추도록.”

Samsung 측 답변 (27:14)

“that was more a comment than a than a question and uh I think we’ll take that into consideration.”

팀에서 pNFS와 flex files도 관리 레이어 후보로 검토 중이며 F_MAP_AP만 사용하는 것이 아니라고 덧붙였다.

2. 쟁점의 핵심: FIEMAP을 DMA 주소 결정에 사용하는 것의 위험성

2.1 F_MAP_AP / FS_IOC_FIEMAP이란?

FS_IOC_FIEMAP은 Linux 커널 ioctl로, 파일의 논리적 바이트 범위를 물리적 블록 주소(extent)로 매핑해 반환한다. AiSIO의 Extend Access Library는 이를 통해 파일의 물리적 LBA를 캐시해두고, GPU가 파일시스템을 거치지 않고 NVMe 커맨드를 직접 발행할 때 이 캐시된 주소를 사용한다.

Christoph Hellwig가 “디버깅 도구”라고 표현한 것은 기술적으로 정확하다. FIEMAP은 원래 e2fsck, filefrag, xfs_bmap 같은 진단·분석 유틸리티 용도로 설계되었으며, 파일시스템은 이 인터페이스에 동시성 보장을 제공하지 않는다.

2.2 TOCTOU 경쟁 조건

FIEMAP이 반환하는 물리적 블록 주소는 호출 시점의 스냅샷이다. 그 이후 어떤 작업이든 블록 주소를 바꿀 수 있다.

t0: FIEMAP 호출 → LBA 1000 반환, 캐시에 저장

t1: 커널, 파일 블록을 LBA 2000으로 이동

t2: GPU가 LBA 1000으로 NVMe Read 발행

결과: 엉뚱한 데이터 읽기, 또는 다른 파일의 데이터 노출

2.3 파일시스템별 위험 요소

- **XFS**: `xfs_swap_extents()`(온라인 디프래그)가 두 파일의 extent를 원자적으로 교환. VFS 이벤트 없이 LBA가 바뀐다. `xfs_scrub` extent 재배치도 동일.
- **ext4**: `ext4_move_extents()` 온라인 디프래그 ioctl. delayed allocation 확정 단계에서 물리 블록 번호 변경 가능.
- **btrfs**: Copy-on-Write 설계로 쓰기 시 항상 새 블록 할당. 백그라운드 balance 작업이 투명하게 블록 이동.
- **공통**: `fallocate PUNCH_HOLE`, `truncate` 후 재쓰기 등.

2.4 EBPf 완화책이 충분하지 않은 이유

AiSIO 팀의 완화책은 “EBPF 트레이스로 파일 변경 알림을 받아 캐시를 무효화한다”는 것이다.

- **경쟁 창 존재:** 혹 실행과 캐시 무효화 사이에 GPU가 NVMe 커맨드를 발행하면 race가 그대로 남는다.
- **감지 불가 이벤트:** xfs_swap_extents(), btrfs balance, xfs_scrub extent 재배치는 VFS 레이어 이벤트를 발생시키지 않아 inotify/fsnotify 기반 혹은 탐지하지 못한다.
- **불안정한 커널 ABI:** 내부 커널 함수에 EBPf 혹은 거는 것은 공식 ABI가 아니다. 커널 버전마다 함수 시그니처가 바뀔 수 있고, upstream 수용 가능성이 낮다.
- **실험 조건의 한계:** 벤치마크가 사전 할당 고정 크기 파일만 사용하므로 문제가 드러나지 않는다. 백그라운드 디프래그가 활성화된 프로덕션에서는 침묵하는 데이터 손상이 발생할 수 있다.

3. 두 접근 방식의 장단점 비교

3.1 Samsung 방식: FIEMAP + EBPf 캐시

장점

- **정상 경로 오버헤드 없음:** 캐시가 유효한 동안은 LBA 조회 없이 GPU가 즉시 NVMe 커맨드를 발행한다. 50M IOPS 벤치마크 결과가 이 구조에서 나온다.
- **즉각적 구현 가능:** 기존 FIEMAP ioctl을 재사용하며, 커널 수정 없이 사용자 공간에서 구현된다.
- **파일시스템 수정 불필요:** XFS/ext4/btrfs 어느 쪽도 건드리지 않는다.
- **ML 훈련 워크로드에서 실질적 안전:** 사전 할당된 고정 크기 파일을 읽기만 하는 경우 블록 재배치가 발생하지 않아 race condition이 현실화되지 않는다.

단점

- **구조적 안전 보장 없음:** TOCTOU race를 소프트웨어 레이어에서 제거할 수 없다. EBPf 알림은 사후 감지(post-hoc)이며 원자성이 없다.
- **VFS 이벤트 없는 재배치 탐지 불가:** xfs_fsr, btrfs balance, xfs_scrub 등이 물리 블록을 옮겨도 혹은 반응하지 않는다.
- **파일시스템마다 동작 상이:** FIEMAP 결과의 의미와 갱신 시점이 파일시스템마다 다르다.
- **커널 upstream 수용 어려움:** 내부 ABI에 의존하는 EBPf 혹은 커널 커뮤니티의 검토를 통과하기 어렵다.
- **워크로드 범위 제한:** 파일이 변경되거나 생성·삭제가 빈번한 일반 워크로드로 확장하기 어렵다.

3.2 Hellwig 권고: pNFS 블록 레이아웃 (RFC 5663)

pNFS는 메타데이터 서버(MDS)와 데이터 서버(DS)를 분리한다. 클라이언트가 직접 I/O를 하려면 MDS로부터 레이아웃 그랜트(layout grant)를 받아야 하며, MDS가 블록을 재배치하려면 클라이언트에게 LAYOUTRECALL을 보내고 응답을 기다려야 한다.

클라이언트 → MDS: LAYOUTGET

MDS → 클라이언트: 레이아웃 그랜트 (범위 → LBA 매핑)

클라이언트: 그랜트 범위 내에서 직접 NVMe DMA

재배치 필요 시: MDS → LAYOUTRECALL → 클라이언트 응답 → 재배치 수행

핵심: FIEMAP은 사후 알림(post-hoc notification), pNFS는 사전 동의(prior consent). 그랜트가 살아있는 동안 파일시스템이 해당 extent를 재배치할 수 없다는 강제적 보장이 있다.

장점

- **구조적 동시성 보장:** lease + recall 메커니즘으로 TOCTOU race를 프로토콜 수준에서 차단한다.
- **파일시스템 독립적:** 프로토콜 레이어 추상화로 XFS/ext4/btrfs 구현 차이를 감춘다.
- **표준 RFC 기반:** 커널 upstream 경로가 명확하고 장기 유지보수 가능성이 높다.
- **클라이언트 보호 내장:** LAYOUTRECALL 핸드셰이크로 재배치 전 진행 중인 I/O 완료를 보장한다.

단점

- **성능 오버헤드:** pNFS는 네트워크 파일시스템 프로토콜로 설계되었다. 로컬 NVMe에 적용하려면 루프백 NFS 스택이 필요하고, 파일마다 MDS 왕복(LAYOUTGET RTT)이 발생한다. Samsung이 보여준 50M IOPS를 full pNFS 위에서 재현하기는 어렵다.

- **LAYOUTRECALL 시 I/O 일시 정지:** 백그라운드 디프래그가 발동하면 GPU I/O가 핸드셰이크 완료 전까지 멈춘다. 레이턴시 스파이크가 발생한다.
- **MDS가 병목 가능:** 대규모 GPU 클러스터에서 수천 개의 파일에 동시에 LAYOUTGET이 몰리면 MDS가 포화된다.
- **구현 복잡도 높음:** XFS, ext4, btrfs 각각에 레이아웃 프로토콜을 구현하고 커널 커뮤니티 리뷰를 통과시키는 것은 수년 단위의 작업이다.

3.3 현실적 중간 지점: 경량 extent lease

Hellwig의 발언을 “full pNFS를 배포하라”로 해석하기보다, “pNFS가 갖춘 lease + recall 시맨틱을 로컬 커널 인터페이스로 구현하라”로 읽는 것이 더 정확하다. 구체적으로는 다음과 같은 방향이다.

- **extent-pin ioctl:** 페이지를 DMA를 위해 핀닝하는 `get_user_pages()`처럼, extent를 DMA 기간 동안 핀닝하는 커널 인터페이스를 추가한다. 핀된 extent는 파일시스템이 재배치할 수 없다.
- **HOMIE가 lease 관리자:** 기존 HOMIE 데몬이 extent 핀 취득·반환을 관리하면 정상 경로는 ioctl 1회로 끝난다. 재배치가 필요할 때만 동기화 비용이 발생한다.
- **파일시스템별 구현 범위:** 전체 pNFS 프로토콜이 아닌 extent-pin/unpin 혹은 각 파일시스템에 추가하면 된다. 구현 범위가 훨씬 작다.

	FIEMAP + EBPf	경량 extent lease	full pNFS
정상 경로 오버헤드	없음	ioctl 1회	LAYOUTGET RTT
재배치 시 오버헤드	없음 (race)	pin 해제 동기화	RECALL 핸드셰이크
동시성 보장	없음	있음	있음
구현 난이도	낮음	중간	매우 높음
Upstream 수용 가능성	낮음	높음	높음

이 방향이 Hellwig가 2년간 제안한 내용의 실체에 가장 가까울 것으로 보인다. Samsung 팀이 구현하지 않은 이유는 기술적 반박보다는 커널 인터페이스 추가에 따르는 커뮤니티 협의와 유지보수 부담을 회피하려는 것으로 보인다.

4. 종합 평가

Hellwig의 비판은 기술적으로 타당하다. FIEMAP 기반 DMA는 구조적 안전 보장이 없고, EBPf 완화책은 VFS 이벤트를 발생시키지 않는 블록 이동을 탐지하지 못한다.

Samsung의 성능 우선 접근도 이해할 수 있다. ML 훈련의 데이터 로딩 워크로드(사전 할당 대용량 파일, 읽기 전용)에서는 race condition이 현실화될 가능성이 매우 낮다. 이 조건에서는 50M IOPS라는 성과를 full pNFS로는 달성하기 어렵다.

그러나 두 입장은 배타적이지 않다. FIEMAP 캐시는 프로토타입과 특정 워크로드 최적화에 유효하지만, 범용 운영 환경을 위해서는 extent lease 시맨틱이 반드시 필요하다. Samsung이 pNFS/flex files를 검토 중이라고 인정한 것은 이 사실을 팀 내부에서도 인식하고 있음을 보여준다.

쟁점	FIEMAP + EBPf	경량 extent lease	full pNFS
동시성 보장	없음	있음	있음
50M IOPS 달성 가능성	있음	대체로 가능	어려움
재배치 시 안전성	경쟁 조건	보장	보장
파일시스템 호환성	파일시스템마다 상이	구현 필요	프로토콜 추상화
Upstream 수용 가능성	낮음	높음	높음
ML 훈련 특화 안전성	실질적으로 안전	안전	안전
일반 워크로드 안전성	위험	안전	안전

AiSIO가 연구 프로토타입을 넘어 범용 운영 환경에 안착하려면 extent lease 메커니즘의 커널 통합이 불가피하다. EBPf 완화책은 그 과도기적 수단으로는 수용 가능하지만 최종 해법이 될 수 없다.